

FaceKey Door Access Control

System Architecture, Workflow Steps & REST API Specification

1. Executive Summary & Overview

FaceKey is an edge-based building door access control platform integrated with the central **Visage** facial recognition server. The system provides real-time face matching, employee identification via relative **InSP numbers** (e.g. InSP/2025/6177/676), automated door authorization checks, and security audit logging.

2. End-to-End Step-by-Step Access Workflow

Step 1: Face Image Capture

Person approaches a door monitored by an edge camera. The camera or client captures the image and sends it along with the target `door_id` to FaceKey endpoint `/access/face-image`.

Step 2: Visage Central Verification

FaceKey sends the image to central Visage backend (https://visage.sltdigitallab.lk/api/face_verification). Visage performs face matching against central database.

Step 3: InSP Employee ID Resolution

Visage returns `face_verified: true`, similarity score (e.g., 0.94), and matched employee **InSP number** (e.g., InSP/2025/6177/676).

Step 4: FaceKey Local Database Lookup

FaceKey queries its local database (users table) to find the employee using the InSP number (supports full InSP string and display name formatting).

Step 5: Authorization Criteria Evaluation

FaceKey checks 6 strict security rules:

- User existence in DB
- User account active (`is_active == true`)
- Face registered (`face_registered == true`)
- Similarity score ≥ 0.60 threshold
- Target door status online (not offline/error)
- Target `door_id` present in `user.authorized_doors` list.

Step 6: Unlock Relay & Audit Logging

If **ALL** rules pass: FaceKey triggers physical relay unlock (`trigger_door_open()`) and writes a GRANTED entry to `access_logs`.

If **ANY** rule fails: Door remains locked and a DENIED log entry is recorded with the exact reason.

3. REST API Endpoints Specification

Base Path: `http://localhost:8000/api/v1/door-access`

Endpoint	Method	Description	Header Required
<code>/access/face-image</code>	POST	Primary flow: Upload face image + door_id & verify via Visage	X-Admin-Id
<code>/access/camera-event</code>	POST	Edge camera flow: Receives pre-recognized InSP ID	X-Admin-Id
<code>/access/check</code>	POST	Manual check: Test user_id authorization for door_id	X-Admin-Id
<code>/doors/{door_id}/open</code>	POST	Admin trigger: Force unlock door manually	X-Admin-Id
<code>/access/otp/request</code>	POST	Fallback flow: Request email OTP for door access	X-Admin-Id
<code>/access/otp/verify</code>	POST	Fallback flow: Verify email OTP & unlock door	X-Admin-Id
<code>/auth/admin/login</code>	POST	Admin authentication (Super Admin / Tenant Admin)	None
<code>/admin/tenants</code>	POST / GET	Tenant management for multi-tenant setup	X-Admin-Id

4. Key Endpoint Request & Response Specifications

Endpoint: **POST `/access/face-image`**

Headers: X-Admin-Id: SUPER001 | Content-Type: multipart/form-data

Form Parameters: - door_id: string (Required, e.g. 'DOOR_001') - camera_id: string (Optional, e.g. 'CAM_MAIN') - image: File upload (Required, JPG/PNG, max 5MB)
Success Response (200 OK - Access Granted): <pre>{ "success": true, "face_verified": true, "access_granted": true, "door_opened": true, "matched_user_id_from_visage": "InSP/2025/6177/676", "visage_similarity_score": 0.94, "door_id": "DOOR_001" }</pre>
Denied Response (200 OK - Door Not Assigned): <pre>{ "success": true, "face_verified": true, "access_granted": false, "door_opened": false, "message": "User not authorized for this door", "matched_user_id_from_visage": "InSP/2025/6177/676" }</pre>

5. Database Schema & Architecture

Table Name	Key Fields	Description
users	id (InSP number), name, email, is_active, face_registered, authorized_doors (JSON array)	Stores employee profiles and assigned door access lists.
doors	id, name, building_id, status (online/offline/error), is_active	Stores door metadata and hardware status.
access_logs	id, door_id, user_id, user_name, event_type (GRANTED/DENIED), similarity_score, details, timestamp	Audit log of all access attempts and door unlock events.
admin_users	id, username, email, password_hash, role (SUPER_ADMIN, TENANT_ADMIN, BUILDING_ADMIN), tenant_id, building_id	Admin user accounts for managing access control permissions.